Architecture as a Variable in LLM-Based Software Generation

Evaluating Message-First Systems

Dvwoo
GitHub: @woozheng

2026

Abstract

Current LLM agent protocols — MCP, A2A, and their contemporaries — follow an additive path: they introduce state management, session handling, orchestration layers, and reliability primitives. This paper proposes an alternative research direction: a *subtractive*, **message-first** design hypothesis.

We ask a design question: when an LLM is not merely writing code *inside* an architecture but generating the architecture itself, what architectural form is most likely to reduce cross-component coordination cost?

We trace a subtractive design path from a multi-agent chat system to the AICP protocol, and show that its endpoint mirrors, at a chosen abstraction level, the way hardware is organized. The hardware analogy is offered as a **structural reference** and **design vocabulary** — not as a proof of superiority, and not as the causal mechanism behind the hypothesis.

We state a falsifiable hypothesis (H1, decomposed into H1a / H1b / H1c) about the *net* effect of message-first architectures on a multi-dimensional cost vector. We propose a 2×2 factorial experimental design to separate the effect of message-first structure from the effect of self-extension.

Whether this framing reveals a meaningful advantage for LLM-native systems remains an empirical question.

Keywords: LLM-native systems, message-driven architecture, subtractive design, architectural cost models, agent protocols, AICP, software generation.

1 Introduction

1.1 Two Paths

The current generation of LLM protocols grows by **addition**.

- **MCP** standardizes tool invocation and resource access.
- **A2A** standardizes inter-agent collaboration.
- **Dapr Agents** and similar infrastructures add reliability primitives.
- **Orchestration frameworks** (LangGraph, CrewAI, Microsoft Agent Framework) add composition.

Each addition solves a real problem. They operate at different layers, and they are not all competitors of one another.

This paper proposes an alternative direction: **subtraction**. Not because addition is wrong, but because a different regime — *the LLM as the primary builder of the system* — may call for different design economics.

1.2 The Shift We Are Interested In

Traditionally, a human designs the architecture and the LLM generates code *within* it.

This paper is interested in a different possibility: what if the architecture itself — the boundaries between components, the flow between them, the contracts at their edges — is also generated and modified by the LLM?

If that is the regime, the design question changes. It is no longer "what architecture do humans find maintainable?" but "what architecture does an LLM find *generatable*, at what *multi-dimensional* cost?"

1.3 Three Propositions

P-struct (structural). AICP can be described with a minimal message structure: plugins are addressable nodes, Envelops are carriers, `route` dispatches.

P-gen (generative). A uniform message schema, a uniform plugin invocation contract, and a uniform routing mechanism may, under specified conditions (§4.2), reduce the cross-component coordination required during system generation and modification.

P-econ (economic). If coordination cost is reduced in a way that is not offset by increases in runtime or maintenance cost, the multi-dimensional engineering cost of message-first architectures may be lower for a specified task class.

P-gen is the bridge that must be built. P-struct can be established by protocol definition. P-econ can be measured by experiment. P-gen must explain why this structure might reduce the generator's actual decision burden, and under what conditions. It must not assume that the burden is eliminated rather than shifted.

AICP is used as a concrete implementation of the message-first architecture family. The paper's research question concerns the family, not the specific implementation.

1.4 Contributions

This paper is a **position paper**. Its contributions are:

1. **A research question.** When the builder of software becomes an LLM, should architecture be treated as an empirical variable rather than a fixed design assumption?
2. **A structural observation.** A message-first architecture can be reached by subtraction, and its endpoint mirrors, at a chosen abstraction level, the way hardware is organized.
3. **A specified mechanism.** Three conditions (contract reuse, local substitutability, discoverable semantics) under which the proposed mechanism is expected to hold. These conditions are not unique to message-first architectures.
4. **A testable hypothesis and method.** H1 (decomposed into H1a / H1b / H1c), stated as a net claim over a multi-dimensional cost vector, with a 2×2 factorial design that separates message-first structure from self-extension and allows the hypothesis to fail.

The value of the paper does not depend on whether H1 is later supported. The research framing — evaluating architecture as an experimental variable for LLM generation — stands regardless of the outcome.

1.5 Scope

H1 applies to *end-to-end generation of a working system from a natural-language requirement, by an LLM, with minimal human intervention.* It is a hypothesis, not a conclusion. It does not follow from any property of LLM I/O; it must be established empirically.

The paper does not claim that message-first is universally better, that state management problems disappear when state moves onto messages, that reducing generation-phase decisions reduces total lifecycle cost, that LLMs cannot reason about objects, or that AICP is functionally superior to existing protocols.

2 A Reference Structure: Hardware

2.1 Three Structural Primitives

At a chosen abstraction level, hardware can be described through three primitives:

- **Nodes**: chips, IP cores, peripherals, memory banks.
- **Messages**: signals, interrupts, bus transactions, packets.
- **Routing**: buses, crossbars, switches, address decoding.

This is a description, not an exhaustive physical account. Hardware also has clocks, power domains, arbitration logic, timing constraints, reset trees, and configuration state.

2.2 Abstraction Level, Not Capability

The claim is not that hardware lacks the ability to express objects, instances, or lifecycles. Hardware has reset, configuration, dynamic device discovery, and resource lifecycle mechanisms, expressed differently from software-level objects and constructors.

The claim is that at the abstraction level we have chosen, hardware is naturally described by three primitives, and that this description is a useful reference structure for comparing software architectures.

2.3 What This Reference Structure Is For

We use the reference structure for two purposes:

1. **Design vocabulary.** Terms (node, message, routing) for describing AICP's minimal architecture.
2. **A concrete structural reference.** Hardware provides an established engineering reference for organizing systems around addressable nodes and communication mechanisms.

The reference makes the structural analogy concrete. It does not establish feasibility, reliability, or superiority of AICP for LLM-generated software.

2.4 Research Motivation, Not Causal Mechanism

"LLM I/O is sequence-shaped" motivates the question "might a message-first architecture be a natural fit?" It does not answer it.

The causal mechanism proposed in this paper is engineering: a uniform message schema and a uniform plugin invocation contract may, under specified conditions, reduce the number of cross-component agreements a generator must keep consistent.

3 Software's Departure from the Reference Structure

3.1 What Software Adds

Software systems employ additional abstractions and mechanisms — such as state machines, isolation, concurrency control, scheduling, and lifecycle management — to satisfy different correctness and operational requirements. These mechanisms are often useful; their necessity and implementation cost depend on the task and on architectural constraints.

3.2 How Mechanisms Interact

Mechanisms tend to interact. State machines often need a place to store state; lifecycle management often interacts with concurrency; concurrency often implies isolation; isolation often interacts with scheduling.

These interactions are not logical necessities. They are patterns that recur in systems that choose to add these mechanisms. The result is that, in many systems, the mechanisms form a cluster: removing one without breaking the others is difficult. This is the **accumulation problem**, noted since at least the microkernel debates of the 1990s.

3.3 The Cost Frame: A Multi-Dimensional Vector

We introduce a multi-dimensional cost vector as the conceptual frame for the hypothesis.

$$C = (C_g,\ C_r,\ C_m)$$

- C_g — *generation cost*: cost of generating architecture, code, interfaces, and flow.
- C_r — *runtime cost*: cost of runtime coordination, error handling, validation, retry, and observability.
- C_m — *maintenance cost*: cost of requirement changes, defect repair, and long-term evolution.

The three dimensions are not the same unit. C_g may be measured in LLM tokens, generation wall-clock time, and human review hours. C_r may be measured in latency, CPU, memory, error rate, and operational hours. C_m may be measured in edit count, repair count, regression-test cost, and human hours.

We do not sum these into a single C_{total}. Doing so would require weights whose origin and sensitivity must be specified.

Reducing C_g does not automatically reduce the vector. A design may shift cost from generation to runtime or from generation to maintenance.

H1 is a net claim: that message-first reduces C_g and/or C_m without a compensating increase in C_r that cancels the benefit. Resource cost (latency, CPU) and human cost (review time, repair time) are reported separately.

3.4 Architectural Decision Constructs

We introduce three constructs, which must be operationalized before they are used as measures:

- **EAC** — *explicit architectural concepts*: the number of distinct architectural mechanisms the generated system directly uses.
- **ADB** — *architectural decision burden*: the number of identifiable cross-module decisions whose consistency is necessary for correctness.

- **DP** — *decision propagation*: the number of modules, contracts, and execution paths affected by a single design change.

ADB is a structural proxy for coordination burden. It measures identifiable cross-module decisions in the generated system, not the LLM's internal reasoning effort.

These three measures are not the same unit and are reported separately, not summed.

Behavioral outcomes — interface error count, first-run acceptance, human intervention count, requirement change cost — are reported independently and are not used to define EAC, ADB, or DP.

3.5 Operationalization: Coding Manual, Independent Coders, Agreement

For EAC and ADB to be reliable, the following are required:

1. A coding manual with category definitions, decision-boundary rules, and worked examples.
2. Two or more independent coders applying the manual to the same set of generated systems.
3. Inter-rater agreement reported using an appropriate statistic.
4. Adjudication of disagreements, with pre-adjudication agreement reported.

The coding procedure is blind: architecture labels are removed from coding materials wherever feasible.

The coding manual, with worked examples, is a required deliverable of the experimental paper that follows.

3.6 If the Measures Cannot Be Reliably Operationalized

If EAC or ADB cannot be reliably operationalized:

- Treat them as a theoretical framework.
- Run experiments on directly observable variables.
- Do not use them as reported primary metrics.

If ADB cannot be reliably measured, H1a is reported as **not measured**, and only H1b and H1c are evaluated empirically.

4 From Design Question to Hypothesis

4.1 The Design Question

> When an LLM generates a complete system — including the boundaries between components, the contracts at their edges, and the flow between them — what architectural form minimizes the multi-dimensional engineering cost, subject to correctness and operational requirements?

The question is empirical.

4.2 The Proposed Mechanism and Its Conditions

Our proposed mechanism is engineering:

> A uniform message schema, a uniform plugin invocation contract, and a uniform routing mechanism may reduce the number of cross-component agreements a generator must keep consistent during generation and modification.

The mechanism is expected to hold under three conditions:

1. **Contract reuse.** The same message and invocation contracts are reused across a substantial fraction of component interactions.
2. **Local substitutability.** A plugin can be added, replaced, or modified without requiring coordinated changes to unrelated components.
3. **Discoverable semantics.** The information needed to select and invoke a plugin is available through explicit contracts or runtime discovery.

These conditions are not unique to message-first architectures. A well-designed object-centric system may also satisfy them.

The empirical question is whether message-first organization makes these conditions cheaper for an LLM to establish and preserve under comparable constraints.

4.3 Architecture Features, Mechanisms, and Outcomes

Layer 1 — Architecture features:

- A single message type (`Envelop`)
- A single plugin invocation contract
- A single routing mechanism (`route`)
- Runtime-generated message flow
- Self-extension (e.g. `create_tool`)

Layer 2 — Proposed mechanisms:

- M1: Fewer cross-module agreements to coordinate
- M2: Smaller interface-change propagation
- M3: Some design decisions can be deferred

Layer 3 — Observable outcomes:

- O1: First-run acceptance rate
- O2: Interface mismatch rate
- O3: Requirement change cost
- O4: Human intervention cost

Terminology:

- *Build-time generation*: plugins, contracts, and flow are generated before execution.
- *Runtime generation*: new plugins or paths are generated during execution.
- *Runtime dispatch*: the existing runtime selects a plugin for a given Envelop during execution.

4.4 Hypotheses

H1 (overall). For end-to-end software generation from natural-language requirements, a message-first architecture with a uniform message schema, a uniform plugin invocation contract, and a uniform routing mechanism may reduce cross-module coordination requirements relative to a capability-matched object-centric baseline, without a compensating increase in runtime or maintenance cost that cancels the benefit.

H1a — Architecture burden. Message-first systems require fewer cross-module coordination decisions (ADB) than a capability-matched baseline. *Requires reliable ADB measurement.*

H1b — Change propagation. Under the same requirement change, message-first systems are affected in fewer modules, contracts, and execution paths (DP) than the baseline.

H1c — Engineering outcomes. Under comparable resource and acceptance criteria, message-first systems have a lower requirement change cost or lower human intervention cost (O3 / O4) than the baseline.

H1a and H1b concern proposed mechanisms; H1c concerns engineering outcomes. Supporting H1a or H1b alone does not establish H1.

Reverse results are findings, not failures. ADB lower with first-run acceptance unchanged, interface mismatch lower with runtime failure higher, or generation cost lower with long-term change cost higher all localize the architectural boundary.

4.5 What H1 Does Not Assume

H1 does not assume that deferred architectural decisions disappear. It assumes only that some decisions can be deferred.

H1 does not assume that message-first is the only way to achieve cross-component uniformity.

5 Subtraction as a Design Method

5.1 Three Kinds of Reduction

Kind	Meaning
Elimination	The problem no longer exists.
Transfer	The problem still exists, but responsibility has been moved.
Deferral	The protocol does not fix a solution; the system chooses one at build time.

Most reductions in AICP are of the second and third kinds. Very few are of the first.

5.2 The Origin: A Multi-Agent Chat System

The design path began with a chat system — a multi-agent collaboration platform with the standard distributed-systems concerns: state management, isolation, concurrency, scheduling, lifecycle. The first version solved these by conventional means.

5.3 Reduction 1: A Unified Envelop

The first reduction introduced a single message type, `Envelop`, and moved execution-relevant state onto it.

This reduction is a transfer, not an elimination. Race conditions, duplicate processing, consistency, coordination, and termination are still concerns. What changes is where state lives and which component owns it.

5.4 Reduction 2: The Agent Becomes Route

In the AICP design, the mandatory central-agent abstraction is replaced by a routing runtime and a set of plugins. Decision-making is assigned to whichever plugin or LLM-enabled component the generated system requires.

This is a design choice within AICP. It is not a claim that all agent architectures must reduce their agents to routers.

5.5 Reduction 3: Everything is a Plugin

Every capability can be given the same invocation contract: a plugin consumes an Envelop and may produce one or more Envelops for subsequent dispatch.

Invocation uniformity is not semantic uniformity. Two plugins with the same invocation contract may have different permissions, resource requirements, failure modes, and security profiles. Uniform invocation does not replace permission control, resource limits, or fault isolation.

5.6 Reduction 4: Agents Are Plugins

The "agent" in the LLM sense is also a plugin. There is one invocation class: the plugin. Behavioral differences between plugins still exist.

5.7 The Endpoint

- Nodes (plugins), identified by a `receiver`.
- Messages (Envelopes), carrying `sender`, `receiver`, `intent`, `payload`.
- Routing (`route`), a lookup by `receiver`.

5.8 Summary Table

Step	Reduction kind	Reduced	Not eliminated
1. Envelop	Transfer	Centralized state ownership	Race conditions, consistency, termination
2. Agent → route	Transfer	Agent as entity	Decision-making (assigned to plugins)
3. Uniform plugins	Deferral	Mandatory plugin-type distinctions	Permission, resource, security semantics
4. Agents as plugins	Reduction of architectural type distinction	Agent/plugin class distinction	Agent-specific behavior may still differ

6 Structural Correspondence

6.1 Core Correspondence

Hardware	AICP	Notes
Node	Plugin	Addressable, independent units
Message	Envelop	Structured in AICP; heterogeneous in hardware
Address	`receiver`	Locating mechanism
Interconnect	`route`	Interconnect vs. software function

6.2 Implementation-Level Analogy

Hardware	AICP
Configuration space	`.contract.json`
Device enumeration	`cogitor`
Driver	Plugin loader
Hot-plug	Hot reload

These are useful for explaining AICP's design but do not add to the core argument.

6.3 Where the Correspondence Breaks Down

- Envelop has structured fields; physical signals are heterogeneous.
- Plugins have reflective access to the runtime; nodes are physical devices.
- Hot reload and hot-plug have different safety constraints.
- Routing in hardware is often determined by physical topology.
- Hardware has clocks, timing, and arbitration that AICP does not model.

6.4 What the Analogy Establishes

The analogy establishes a structural vocabulary for describing AICP's minimal architecture, and a concrete reference showing that message-first organization is engineering-mature in at least one domain.

It does not establish that message-first is superior for LLM generation, that AICP's implementation is feasible at hardware scale, or that the analogy is the causal mechanism behind H1.

7 The LLM Within the Message Structure

7.1 main_agent as Shell

`main_agent` is a plugin. Its `execute` follows the same invocation contract as any other plugin.

7.2 The LLM as Decision-Maker

Each round, the LLM produces one of: `use_tool`, `reply`, `create_tool`, `fix_tool`, `remove_tool`. The system does not know them in advance.

7.3 Self-Extension

When the LLM chooses `create_tool`, the system produces a new plugin, compiles it, and registers it. The LLM can then invoke it using the same schema.

7.4 The Flow as Externalized Trace

The `InformationFlow` is an externalized interaction trace and a source of working context for the LLM. The effective context available in any round depends on retrieval, summarization, and context-window limits; it is not the entire flow.

8 From Protocol Minimalism to AI-Generated Systems

8.1 The Central Question

> When the primary builder of a software system shifts from a human programmer to an LLM, should the architecture shift from object-centric to message-centric?

8.2 What a Message-First Protocol Provides

- A single message type (`Envelop`)
- A single plugin invocation contract
- A single routing mechanism (`route`)
- A single discovery mechanism (`cogitor`)
- A single contract mechanism (`contract_agent`)

8.3 The Distinction from Traditional Architectures

In traditional architectures, workflows and controllers are typically designed ahead of time.

In a message-first architecture with an LLM at its center, business-level orchestration can be generated at build time by the LLM. The runtime still has fixed routing, execution, and safety mechanisms. What is not fixed is which plugins exist and what paths they take.

8.4 From Architectural Generation to Executable Validation

Architectural generation is not the same as correct system generation. We separate four stages:

Stage	**What happens**	**Failure mode**
1. Architectural generation	Plugin boundaries, contracts, message paths	Incoherent boundaries
2. Code generation	Plugin implementations	Syntax errors, wrong types
3. Execution	Message dispatch	Runtime errors, deadlocks
4. Acceptance validation	Tests confirm requirements	Functional mismatches

AICP's contribution is not to guarantee any single stage. It provides a uniform structure in which the four stages can iterate.

9 Why This Was Not Common Before

9.1 The Human Bottleneck

A message-first system with runtime-generated plugins and self-extension would be costly to maintain by humans.

9.2 The LLM's Fit

LLMs can perform tasks that reduce this cost: reading long message flows, deciding next actions, extracting contracts, generating plugins, repairing plugins. These capabilities depend on context length, retrieval, tools, and validation mechanisms.

9.3 An Economic Hypothesis

Message passing, Actor models, microkernels, event-driven systems, and plugin architectures predate LLMs by decades. The paper does not claim that message-first architectures were impossible before LLMs.

The hypothesis is that architectural choices historically optimized for human design and maintenance may not minimize the cost of LLM-driven generation and evolution. This is a hypothesis about engineering economics, not a claim that object-centric architectures are inherently less legible to LLMs.

10 Experimental Design

10.1 Task Class

End-to-end generation of a working system from a natural-language requirement, by an LLM, with minimal human intervention.

10.2 2×2 Factorial Design

	Object-centric	**Message-first**
Static	A1	A2
Dynamic	B1	B2

Factor A — architecture family.

- **Object-centric baseline**: components organized around objects, service interfaces, and explicit call relationships. Mature interfaces, dependency injection, and plugin mechanisms are permitted.
- **Message-first**: the primary component-interaction mechanism is a uniform `Envelop`, a uniform plugin invocation contract, and `receiver` routing.

Factor B — self-extension.

- **Static**: no runtime plugin generation.
- **Dynamic**: runtime plugin generation enabled.

Both architectures must have a fair implementation opportunity for self-extension. Self-extension is not a capability unique to AICP.

Architecture-constraint adherence must be recorded. If the LLM in the object-centric group generates extensive uniform-message mechanisms, or the message-first group generates special bypasses, the result is reported with this deviation noted, not treated as a clean comparison.

Comparisons enabled:

- A2 vs. A1: independent effect of message-first structure.
- B1 vs. A1: value of self-extension under object-centric organization.
- B2 vs. A2: value of self-extension under message-first organization.
- Interaction between architecture and self-extension.

10.3 Dependent Variables

EAC, ADB, DP, and the three cost dimensions are reported separately, not summed.

Variable	Layer	Definition
First-run acceptance rate	O1	Fraction of tasks where, after initial generation with no human code edits, the system passes all pre-defined acceptance tests.
Interface mismatch rate	O2	Cross-module contract mismatches causing test failures.
Requirement change cost	O3	Edits, time, or token cost to add a specified requirement after initial acceptance.
Human intervention cost	O4	Manual edits, or human work time, to reach acceptance.
EAC	L1	Number of distinct architectural mechanisms used.
ADB	L2	Number of independent cross-module coordination decisions.
DP	L2	Number of modules, contracts, paths affected by a single design change.
C_g / C_r / C_m	Cost	Generation / runtime / maintenance cost, reported separately.

10.4 Controls

- Same model version, temperature, tool permissions.
- Same task descriptions, acceptance criteria, execution environment.
- Same context budget and generation count.
- Same debugging, retry, and human intervention policy.
- Matched infrastructure support; separate cost accounting for generated system vs. pre-supplied infrastructure.
- Self-extension implementation matched across architecture families in the dynamic condition.

10.5 Task Set and Primary Metrics

Task	Description	Primary metric
A. Simple business flow	Input validation → processing → response	First-run acceptance rate (O1)
B. Dynamic branching and multi-plugin collaboration	Branching, parallel branches, failure recovery	Interface mismatch rate (O2)
C. Requirement change after acceptance	Add a capability to a tested system	Requirement change cost (O3)

10.6 Statistical Units and Task Diversity

- **Task**: the abstract specification.
- **Run**: one generation-and-evaluation cycle on one task with one configuration.
- **System**: the final generated system, evaluated once.

Task diversity and repetition count are distinct dimensions. Multiple runs on the same task estimate generation-process randomness; different tasks estimate generalization across task types. Both are reported.

Analysis is at the run level, with task as a stratifying factor.

10.7 Pre-Registration, Non-Inferiority, and Reporting

Pre-registered outcome criteria. For each sub-hypothesis, we pre-specify the primary metric, the expected direction, and the minimum practical effect size.

Non-inferiority for the cost vector. "No significant difference" is not "evidence of equivalence." If the hypothesis is that runtime cost does not increase materially, a non-inferiority margin is pre-specified and a non-inferiority test is used. The margin must be defined before the experiment.

Outcome labels per sub-hypothesis:

- **Supported**: pre-registered primary prediction met with sufficient evidence and practical significance.
- **Not supported / inconclusive**: evidence insufficient, or effect too small.
- **Evidence against**: pre-registered primary result shows a practically significant reverse effect.

Relationship to the overall hypothesis: H1 is reported as supported only if H1c (or its pre-registered dominant outcome) is met and the cost vector does not show a compensating increase beyond the pre-specified non-inferiority margins.

Generation failure is a result. First-run acceptance and subsequent change cost are reported separately. Runs that fail to produce an acceptable system are not silently dropped.

10.8 Sample Size

Run count is pre-specified. Where feasible, a power analysis is conducted using pilot variance. Where power analysis is not feasible, we pre-specify a conservative run count and report it as a limitation.

Pre-registered before execution: runs per condition; per-task run count; whether repeated runs share model state or cache; handling of failures, timeouts, missing results; multiple-comparison correction; confidence-interval procedures and practical-significance thresholds.

10.9 What the Experiment Can and Cannot Show

Can show: whether first-run acceptance, interface mismatch, change cost, and human intervention differ; whether self-extension contributes independently; whether the effect differs by architecture family.

Cannot show: that message-first is universally better; that observed differences are due solely to a single architectural primitive; that the architectural decision constructs are a unified metric.

10.10 Reliability Precondition

Before the architectural measures are reported: a coding manual must exist; two or more independent coders must apply it; inter-rater agreement must be reported. If reliability cannot be demonstrated, H1a is reported as not measured.

11 Related Work

11.1 Message-Passing and Modular Systems

The Actor model (Hewitt 1973; Agha 1986), Erlang (Armstrong 2007), microkernels (Liedtke 1995), event-driven architectures (Fowler 2005), and plugin architectures predate this paper by decades.

AICP inherits from these traditions. Its originality is not the proposal of message passing. Its originality is the application of message-first design as a variable in LLM system generation.

11.2 LLM Agent Protocols and Orchestration Frameworks

- **MCP** standardizes tool invocation and resource access.
- **A2A** standardizes inter-agent collaboration.
- **Dapr Agents** add durability and reliability primitives.
- **Orchestration frameworks** compose flows.

These operate at different layers and are not all direct competitors of AICP.

11.3 LLM Software Generation, Auto-Repair, and Architecture Evaluation

The central question is not "is message passing mature?" but "does architecture organization affect the cost of LLM generation and evolution of a complete system?" This places the paper in the same research area as work on:

- LLM code generation benchmarks and evaluation.
- Automatic program repair.
- Software architecture evaluation.

The methodological contribution of this paper is to add architecture organization as an experimental variable, alongside the model and the task. Whether prior work has already addressed this specific question is a matter for the extended version of this paper.

11.4 What AICP Adds as a Research Question

> When the builder of the architecture is an LLM, does architecture choice become an experimental variable, and does message-first design reduce the multi-dimensional cost of generation and evolution relative to a capability-matched baseline?

12 Discussion

12.1 The Research Framing

The research framing is new in its emphasis: message-first architecture is treated as an explicit experimental variable in LLM-driven system generation. The underlying architectural ideas have a long history. Whether this framing reveals a meaningful advantage for LLM-native systems remains an empirical question.

12.2 The Target Is Moving

LLMs themselves are evolving. Context windows grow. Reasoning improves. Tool use stabilizes. As LLMs change, the boundary of what is natural for an LLM to generate changes with them.

It is possible that, as LLMs develop, object-centric architectures become as natural for them to generate as message-first ones. It is also possible that the opposite happens. The paper assumes only that the question is worth asking, and that the method for asking it should be explicit.

12.3 What Would Falsify the Hypothesis

If message-first systems show higher interface mismatch rate under controlled conditions, H1b is not supported. If they show higher change cost, H1c is not supported. If the cost vector shows a compensating increase in runtime or maintenance cost, H1 overall is not supported.

A negative result localizes the architectural boundary, which is itself useful information.

12.4 An Invitation

The paper does not ask the reader to accept that message-first is better. It asks the reader to consider that architecture should be evaluated differently when the builder changes — and to test whether a specific structural form reduces the cost of generation and evolution.

12.5 Threats to Validity

Construct validity. ADB is a structural proxy for coordination burden, not a direct measurement of cognitive effort. If the coding manual cannot be reliably applied, the construct is not measured.

Internal validity. LLM behavior is stochastic. Multiple runs, pre-registered criteria, and matched infrastructure reduce confounds. Randomness remains a source of variance.

External validity. The task set is small and chosen to probe specific dimensions. Results may not generalize to all software domains.

Ecological validity. The tasks are generated and evaluated in a research setting, and may not reflect the complexity and long-horizon evolution of real development environments.

These threats are named, not resolved. The experimental protocol is designed to make them visible in the results.

13 Conclusion

This paper proposes a research framing in which message-first architecture is treated as an explicit experimental variable in LLM-driven system generation, and states a falsifiable hypothesis about its net effect on a multi-dimensional cost vector.

The contributions are a research question, a structural observation, a specified mechanism, and a testable method. AICP is used as a concrete implementation of the message-first family.

The broader research question remains open:

> When the primary builder of software becomes an LLM, should architecture shift from object-centric to message-centric?

The paper does not answer it. It proposes how to find out. And it acknowledges that the answer may change as LLMs themselves continue to evolve.

A The Subtraction Table

B Sub-Hypotheses and Reporting

Each is reported independently, with **Supported** / **Not supported** / **Inconclusive** / **Evidence against** as outcomes.

H1 is supported only if its pre-registered dominant outcome is met and the cost vector does not show a compensating increase beyond the pre-specified non-inferiority margins.

References

[1] Agha, G. (1986). *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press.

[2] Armstrong, J. (2007). *Programming Erlang*. Pragmatic Bookshelf.

Step	Reduction kind	Reduced	Not eliminated
0. Chat system	—	—	State machine, isolation, concurrency, scheduling
1. Envelop	Transfer	Centralized state ownership	Race conditions, consistency, termination
2. Blind router	Transfer	Agent as entity	Decision-making (assigned to plugins)
3. Uniform plugins	Deferral	Mandatory plugin-type distinctions	Permission, resource, security semantics
4. Agents as plugins	Reduction of architectural type distinction	Agent/plugin class distinction	Agent-specific behavior may still differ

Sub-hypothesis	Mechanism	Outcome	Requires
H1a	Fewer cross-module coordination decisions (ADB)	—	Reliable EAC/ADB measurement
H1b	Smaller change propagation (DP)	—	Standard change set
H1c	—	Lower change cost (O3) / lower human intervention (O4)	Pre-registered criteria

[3] Dvwoo (2026). *AICP: A Minimal Protocol for LLM-Native Agent Systems*. GitHub: woozheng/aicp.

[4] Dvwoo (2026). *Protocol as Neural: Toward a Protocol-Centric General Artificial Intelligence Operating System*. GitHub: woozheng/aicp.

[5] Dvwoo (2026). *AICP Protocol Specification v6.0*. GitHub: woozheng/aicp.

[6] Fowler, M. (2005). *Event Sourcing*. martinfowler.com.

[7] Hewitt, C. (1973). A Universal Modular Actor Formalism for Artificial Intelligence. *IJCAI*.

[8] Liedtke, J. (1995). On μ-Kernel Construction. *SOSP*.

[9] Milner, R. (1999). *Communicating and Mobile Systems: the π-Calculus*. Cambridge University Press.

[10] Anthropic (2025). *Model Context Protocol Specification*. modelcontextprotocol.io.

[11] Google / Linux Foundation (2025). *Agent-to-Agent (A2A) Protocol Specification*. a2aproject.org.

[12] Dapr (2026). *Dapr Agents Documentation*. docs.dapr.io.